LinStabPy: a framework for compressible stability and resolvent analysis

Gabriel Y. R. Hamada[1,2], Ricardo A. S. Frantz[3]

[1] School of Mechanical Engineering, University of Campinas, Campinas, SP 13083-860, Brazil

[2] DynFluid, Arts et Métiers Paris / CNAM, 151 Bd de l'Hôpital, 75013 Paris, France

[3] Laboratory of Fluid Mechanics and Instabilities,
École Polytechnique Fédérale de Lausanne, CH-1015 Lausanne, Switzerland

Abstract

We present LinStabPy, an open Python framework for compressible linear stability and resolvent analysis built around a shared discrete operator for supplied base or mean flows. The discretization employs summation-by-parts finite differences on curvilinear grids; the resulting sparse systems are solved through PETSc/SLEPc. A software layer reuses the same assembled operator across temporal stability, spatial stability, and resolvent workflows. Independent operator checks, fidelity assessment against the reference MATLAB implementation, and validation in an oblique shock-wave/boundary-layer-interaction setting are collected in the Supplementary Material. The implementation is released as the Python package `linstabpy`.

Keywords: compressible stability; resolvent analysis; linearized Navier–Stokes; sparse solvers; PETSc; SLEPc

1 Introduction

Linear stability and resolvent analyses remain standard tools for compressible-flow problems once a reference state is already available from simulation, experiment, or a separate base-flow solver [1–4]. In this setting the central object is the discrete linearized operator, which—rather than the base-flow generator itself—supports modal, nonmodal, and frequency-domain analyses. The practical bottleneck is therefore twofold. One needs a trustworthy compressible discretization, and one needs a solver stack that remains usable when the resulting sparse systems become large.

LinStabPy builds on two public repositories: the `LinStab2D` repository [5, 6] and the `resolvent4py` repository [7–9]. The former provides the compressible formulation on structured and curvilinear grids, including summation-by-parts finite differences, boundary-condition recipes, and resolvent analysis. The latter introduces reusable operator abstractions for large-scale linear analyses. LinStabPy combines these layers: a compressible discrete operator for supplied flows, an analysis interface written in terms of that operator, and interchangeable PETSc/SLEPc solver backends for direct and iterative linear-algebra workflows.

The surrounding software ecosystem is expanding. Packages such as `BROADCAST` [10], `dNami` [11], and `LightKrylov` [12] reflect growing open-source activity around operator-based workflows and Krylov-based numerics; see Frantz et al. [13] for a review of Krylov methods in fluid-dynamics applications.

LinStabPy addresses a narrower workflow than these broader packages. Many users can obtain a nonlinear base or mean flow but lack a ready-to-use compressible linear operator with the needed thermodynamic, curvilinear, and boundary-condition support. LinStabPy fills that gap: the base or mean flow is obtained from the user's own CFD or data-generation environment, and the package discretizes the corresponding linearized operator for prototyping and production-scale stability/resolvent calculations.

The main contribution is the framework itself: rather than reporting new flow-physics findings, we provide a computational tool that turns supplied compressible reference states into a discrete operator reusable across temporal stability, spatial stability, and resolvent analyses. The detailed formulation, verification protocol, and benchmark results are collected in the Supplementary Material.

2 Framework

The framework takes as input a compressible base or mean flow together with the mesh, finite-difference order, boundary-condition recipe, and filter choices. Together, these define a fully discrete linearized operator. Temporal stability, spatial stability, and resolvent analysis then act on that shared operator or on derived

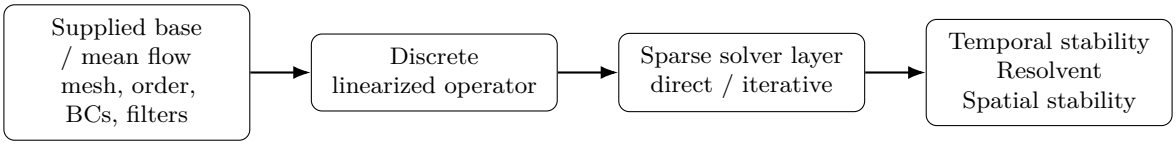


Figure 1: LinStabPy framework overview. User-supplied inputs define a discrete linearized operator, which feeds stability and resolvent workflows through a direct or iterative sparse solver layer.

shifted/block variants, while the sparse linear algebra is handled through PETSc/SLEPc [14, 15] with MUMPS direct factorization [16] for eigenvalue problems and either direct or iterative Krylov/preconditioning routes [17, 18] for resolvent analyses.

Because the operator is assembled explicitly as a sparse matrix, it pairs naturally with direct sparse-LU factorization for the shift-invert inner solves that eigenvalue problems require. This is distinct from the time-stepper approach [19, 20], in which the linearized operator is never formed and Krylov methods serve as the *outer* eigensolver acting on propagator snapshots; see Frantz et al. [13] for a review. The two approaches use "Krylov methods" in different roles (outer iteration on the propagator versus inner iteration on a shifted system), and the distinction matters for solver selection: iterative inner solves encounter the non-normality of the shifted operator directly, whereas the time-stepper outer loop avoids inversion altogether. The Supplementary Material (Section S6) discusses this point in detail.

The framework separates physical assembly from operator analysis: changing the finite-difference order, coordinate mapping, or boundary recipe affects only the operator-construction step, not the downstream analysis logic. This design builds on the compressible formulation of Martini and Schmidt [6] while providing a reusable software layer for current and future operator-based methods, including the harmonic-resolvent extensions discussed in Padovan et al. [21], Padovan and Rowley [22], Farghadan et al. [23].

3 Verification and scope

The Supplementary Material presents verification evidence in three parts: two independent checks of the assembled discrete operator, fidelity assessment through the five shipped Martini examples [6], and validation in the oblique SWBLI setting of Robinet [24] and Song and Hao [25]. It also contains the compressible linearized Navier–Stokes derivation, curvilinear and SBP discretization details, solver configuration, the discrete Chu energy norm, and the harmonic-resolvent extension.

4 Code availability

LinStabPy is available at `https://github.com/ricardofrantz/LinStabPy` as the Python package `linstabpy`; release DOI and snapshot metadata will be added in the final submission package.

References

[1] Peter J. Schmid and Dan S. Henningson. *Stability and Transition in Shear Flows*. Springer, 2001.

[2] Vassilis Theofilis. Global linear instability. *Annual Review of Fluid Mechanics*, 43:319–352, 2011.

[3] Beverley J. McKeon and Ati Sharma. A critical-layer framework for turbulent pipe flow. *Journal of Fluid Mechanics*, 658:336–382, 2010.

[4] Aaron Towne, Oliver T. Schmidt, and Tim Colonius. Spectral proper orthogonal decomposition and its relationship to dynamic mode decomposition and resolvent analysis. *Journal of Fluid Mechanics*, 847: 821–867, 2018.

[5] LinStab2D repository. `https://github.com/LinStab2D/LinStab2D`, 2026. GitHub repository, accessed 2026-03-13.

[6] Eduardo Martini and Oliver Schmidt. Linstab2D: stability and resolvent analysis of compressible viscous flows in MATLAB. *Theoretical and Computational Fluid Dynamics*, 38(5):665–685, 2024. doi: 10.1007/s00162-024-00706-0.

[7] resolvent4py repository. `https://github.com/albertopadovan/resolvent4py`, 2026. GitHub repository, accessed 2026-03-13.

[8] Alberto Padovan, Vishal Anantharaman, Clarence W. Rowley, Blaine Vollmer, Tim Colonius, and Daniel J. Bodony. Resolvent4py: An open-source package for stability, resolvent and modal decomposition analyses in python. *SoftwareX*, 31:102286, 2025. doi: 10.1016/j.softx.2025.102286.

[9] Alberto Padovan. resolvent4py example gallery: Cgl. `https://albertopadovan.github.io/resolvent4py/auto_examples/cgl/index.html`, 2026. Accessed 2026-03-01.

[10] Arthur Poulain, Cédric Content, Denis Sipp, Georgios Rigas, and Eric Garnier. BROADCAST: A high-order compressible CFD toolbox for stability and sensitivity using algorithmic differentiation. *Computer Physics Communications*, 283:108557, 2023. doi: 10.1016/j.cpc.2022.108557.

[11] Nicolas Alferez, Emile Touber, Stephen Winn, and Yussuf Ali. dnami: a framework for solving systems of balance laws using explicit numerical schemes on structured meshes. `https://doi.org/10.5281/zenodo.6720593`, 2022. Zenodo software release.

[12] J. Simon Kern, Ricardo S. Frantz, and Jean-Christophe Loiseau. LightKrylov: Lightweight implementation of Krylov subspace techniques in modern Fortran. *Journal of Open Source Software*, 11(117):9623, 2026. doi: 10.21105/joss.09623.

[13] Ricardo A. S. Frantz, Jean-Christophe Loiseau, and Jean-Christophe Robinet. Krylov methods for large-scale dynamical systems: Application in fluid dynamics. *Applied Mechanics Reviews*, 75(3):030802, 2023. doi: 10.1115/1.4056808.

[14] Satish Balay et al. Petsc/tao users manual. *Argonne National Laboratory*, 2025. ANL-95/11 - Revision 3.24.

[15] Vicente Hernandez, Jose E. Roman, and Vicente Vidal. SLEPc: A scalable and flexible toolkit for the solution of eigenvalue problems. *ACM Transactions on Mathematical Software*, 31(3):351–362, 2005.

[16] Patrick R. Amestoy, Iain S. Duff, Jacko Koster, and Jean-Yves L'Excellent. A fully asynchronous multifrontal solver using distributed dynamic scheduling. *SIAM Journal on Matrix Analysis and Applications*, 23(1):15–41, 2001.

[17] Yousef Saad. *Iterative Methods for Sparse Linear Systems*. SIAM, 2 edition, 2003.

[18] Michele Benzi. Preconditioning techniques for large linear systems: a survey. *Journal of Computational Physics*, 182(2):418–477, 2002.

[19] W. S. Edwards, Laurette S. Tuckerman, Richard A. Friesner, and Danny C. Sorensen. Krylov methods for the incompressible Navier–Stokes equations. *Journal of Computational Physics*, 110(1):82–102, 1994.

[20] Dwight Barkley, Hugh M. Blackburn, and Spencer J. Sherwin. Direct optimal growth analysis for timesteppers. *International Journal for Numerical Methods in Fluids*, 57(9):1435–1458, 2008.

[21] Alberto Padovan, Samuel E. Otto, and Clarence W. Rowley. Analysis of amplification mechanisms and cross-frequency interactions in nonlinear flows via the harmonic resolvent. *Journal of Fluid Mechanics*, 900:A14, 2020. doi: 10.1017/jfm.2020.497.

[22] Alberto Padovan and Clarence W. Rowley. Analysis of the dynamics of subharmonic flow structures via the harmonic resolvent: application to vortex pairing in an axisymmetric jet. *Physical Review Fluids*, 7 (7):073903, 2022. doi: 10.1103/PhysRevFluids.7.073903.

[23] Ali Farghadan, Junoh Jung, Rutvij Bhagwat, and Aaron Towne. Efficient harmonic resolvent analysis via time stepping. *Theoretical and Computational Fluid Dynamics*, 38(3):331–353, 2024. doi: 10.1007/s00162-024-00694-1.

[24] Jean-Christophe Robinet. Bifurcations in shock-wave/laminar-boundary-layer interaction: global instability approach. *Journal of Fluid Mechanics*, 579:85–112, 2007. doi: 10.1017/S0022112007005095.

[25] Ziming Song and Jiaao Hao. Global instability of the interaction between an oblique shock and a laminar boundary layer. *Physics of Fluids*, 35(8), 2023. doi: 10.1063/5.0160648.